

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет Безопасности информационных технологий
Образовательная программа Технологии защиты информации
Направление подготовки(специальность) 10.03.01 Информационная безопасность

О Т Ч Е Т
об учебной практике

Тема задания: Разработка сценариев тестирования и оценка защищённости контроля доступа в Attack-Defense CTF

Обучающийся:

студентка группы N3445 Доан Тхи Хоай Тхыонг

Согласовано:

Руководитель практики в ИТМО: Менщиков Александр Алексеевич, доцент ФБИТ, к.т.н., доцент, декан ФБИТ, директор учебно-практического центра - Киберполигон

Ответственный за практику от университета: Еритенко Николай Алексеевич, инженер ФБИТ

Практика пройдена с оценкой_____

Дата 13.02.2026

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Содержание.....	2
Введение.....	3
1 Основы Attack-Defense CTF и принципы тестирования контроля доступа.....	4
1.1 Формат Attack-Defense CTF и критерии работоспособности.....	4
1.2 Контроль доступа в веб-сервисах: аутентификация и авторизация.....	5
1.3 Типовые уязвимости и логические ошибки контроля доступа и их последствия.....	7
1.4 Подход к тестированию контроля доступа в условиях Attack-Defense.....	8
1.5 Инструменты и артефакты проверки.....	9
2 Разработка тест-сценариев и оценка защищённости контроля доступа в условиях Attack-Defense CTF.....	11
2.1 Условия соревнования и стенд эксперимента.....	11
2.2 Краткое описание сервиса и модели данных.....	11
2.3 Поверхность атаки и точки контроля доступа.....	11
2.4 Анализ логических ошибок контроля доступа.....	12
2.5 Разработка сценариев тестирования и тест-матрицы.....	13
2.6 Рекомендации и валидация.....	23
Заключение.....	26
Список использованных источников.....	27

ВВЕДЕНИЕ

В формате соревнований Attack-Defense CTF команды одновременно поддерживают работоспособность собственных сервисов и противодействуют атакам соперников, стремящихся похитить приватные данные пользователей. В качестве таких данных в A/D выступают флаги - строки заданного формата, которые размещаются в сервисах автоматическими чекерами. Поэтому ключевым фактором успешного выступления является баланс между устойчивостью сервиса (SLA) и защищённостью приватных данных, поскольку утечки флагов приводят к потере очков, а сбои функциональности - к статусам MUMBLE/CORRUPT/DOWN и снижению SLA.

Одной из наиболее распространённых причин компрометации приватности в веб-сервисах являются логические ошибки контроля доступа. Даже при корректной аутентификации недостатки авторизации на уровне объектов (например, IDOR/BOLA), несогласованные проверки прав в разных эндпоинтах, ошибки обработки сессий и некорректные условия доступа могут позволить атакующему получить доступ к чужим данным без явного взлома инфраструктуры. В условиях Attack-Defense такие дефекты особенно критичны: они, как правило, дают быстрый и масштабируемый способ извлечения флагов, при этом часто не нарушая видимую работоспособность сервиса.

Целью данной работы является разработка сценариев тестирования контроля доступа и оценка защищённости веб-сервиса в условиях Attack-Defense CTF. Для достижения цели рассматриваются особенности формата A/D (раунды, чекеры, статусы и SLA), анализируются типовые логические уязвимости контроля доступа, формируется подход к построению тестов с критериями pass/fail и приоритизацией по риску утечки флагов, а также определяется набор инструментов и артефактов, необходимых для воспроизводимого тестирования. Практическая часть основывается на развёрнутом стенде (vulnbox/Docker) и включает построение карты поверхности атаки, разработку тест-матрицы, проведение проверок и формирование рекомендаций, не ухудшающих работоспособность сервиса и сценарии чекера.

Результаты работы могут быть использованы как в подготовке к соревнованиям Attack-Defense CTF, так и при прикладной проверке веб-сервисов на предмет уязвимостей контроля доступа с учётом требований к стабильности и корректной функциональности.

1 **ОСНОВЫ ATTACK-DEFENSE CTF И ПРИНЦИПЫ ТЕСТИРОВАНИЯ КОНТРОЛЯ ДОСТУПА**

1.1 **Формат Attack-Defense CTF и критерии работоспособности**

Attack-Defense CTF - это соревнование в реальном времени, в котором каждая команда обслуживает одинаковый набор сетевых сервисов и одновременно выполняет две задачи:

1. защищать свои сервисы от атак соперников;
2. атаковать сервисы других команд, чтобы извлекать приватные данные.

Ключевая особенность формата: уязвимости и логика сервисов одинаковы у всех команд, поэтому найденная ошибка в собственном экземпляре сервиса с высокой вероятностью применима к сервисам соперников.

Раунд - фиксированный промежуток времени, в конце которого сервисы проверяются автоматическими программами организаторов - чекерами. Чекер имитирует действия обычного пользователя и выполняет типовой набор операций: регистрация/вход/создание данных/чтение данных/проверка ранее сохранённых данных. В приватные данные чекер помещает флаги - строки фиксированного формата, обычно удовлетворяющие регулярному выражению:

`[A-Z0-9]{31}=`

Флаг трактуется как конфиденциальная информация, которая должна быть доступна владельцу по логике приложения, но недоступна другим пользователям. В A/D именно утечка флагов равнозначна потере приватности.

По результатам проверки чекер присваивает сервису один из статусов:

- OK - сервис доступен и корректно выполняет сценарии чекера;
- MUMBLE - сервис доступен, но часть функциональности нарушена (ошибки бизнес-логики, неверные ответы, поломка отдельных операций);
- CORRUPT - нарушена целостность/доступность пользовательских данных: ранее сохранённые данные (в т.ч. флаги) не удаётся получить владельцу;
- DOWN - сервис недоступен по сети или не отвечает в разумное время.

На основании статусов рассчитывается SLA (Service Level Agreement) сервиса - доля раундов, в которых сервис был в работоспособном состоянии (обычно учитываются раунды со статусом OK, иногда OK+MUMBLE - в зависимости от правил конкретного соревнования). SLA является критичным показателем, поскольку даже при успешных атаках низкий SLA резко снижает итоговый результат команды.

Помимо SLA учитываются очки за флаги (FP): если команда извлекает флаги с чужих сервисов и корректно сдаёт их в систему жюри, она получает FP, а команда-жертва теряет очки. Итоговый счёт в большинстве A/D-соревнований зависит от сочетания FP и SLA, поэтому стратегия «ломать сервис» обычно невыгодна: падение SLA снижает ценность любых полученных очков.

UP		CORRUPT		MUMBLE		DOWN		CHECK FAILED	
#	team	score	Zapiski				otkritki		
1	Кабанчики 172.25.2.2	5000.00	SLA : 100.00%		●	SLA : 100.00%		●	
			FP : 2500.00			FP : 2500.00			
			🚩 +0/-0			🚩 +0/-0			
2	ООО ПЕНЕТРЕЙТОР\$\$ 172.25.1.2	5000.00	SLA : 100.00%		●	SLA : 100.00%		●	
			FP : 2500.00			FP : 2500.00			
			🚩 +0/-0			🚩 +0/-0			
3	SSH 172.25.5.2	4945.05	SLA : 98.90%		●	SLA : 98.90%		●	
			FP : 2500.00			FP : 2500.00			
			🚩 +0/-0			🚩 +0/-0			
4	Ливерпуль 172.25.4.2	4917.58	SLA : 100.00%		●	SLA : 96.70%		●	
			FP : 2500.00			FP : 2500.00			
			🚩 +0/-0			🚩 +0/-0			
5	WELCOME 172.25.3.2	4815.65	SLA : 94.51%		●	SLA : 92.31%		●	
			FP : 2653.77			FP : 2500.00			
			🚩 +2/-0			🚩 +0/-0			
6	NPC 172.25.6.2	4712.24	SLA : 97.80%		●	SLA : 96.70%		●	
			FP : 2346.23			FP : 2500.00			
			🚩 +0/-2			🚩 +0/-0			

Рисунок 1 - Итоговая рейтинговая таблица второго соревнования

1.2 Контроль доступа в веб-сервисах: аутентификация и авторизация

Контроль доступа - совокупность механизмов, обеспечивающих:

- идентификацию пользователя и установление его личности;
- проверку прав на конкретные действия и объекты данных;
- фиксацию событий доступа (аудит, журналы).

В рамках веб-сервисов различают два базовых компонента:

Аутентификация (Authentication) отвечает на вопрос: «Кто ты?»

Это процесс подтверждения личности пользователя перед тем, как выдать доступ (сессию/токен). Типовые механизмы:

- логин/пароль (с хранением пароля в виде хэша и безопасной проверкой);
- cookie-сессии (сервер выдаёт cookie, по которому идентифицирует пользователя);
- токены (например, JWT), API-ключи;
- многофакторная аутентификация (реже в CTF, но типична в реальных системах).

Авторизация (Authorization) отвечает на вопрос: «Что тебе можно?»

Это проверка прав уже аутентифицированного пользователя на конкретное действие или объект: чтение, изменение, удаление, создание.

На практике авторизация строится на:

- RBAC (Role-Based Access Control): доступ на основе ролей (user/admin и т.п.);
- ABAC (Attribute-Based Access Control): доступ зависит от атрибутов пользователя/объекта/контекста;
- Ownership-подход: пользователь имеет доступ только к объектам, которыми владеет или в которых участвует (например, отправитель/получатель).

Ключевой принцип для веб-сервисов:

все проверки контроля доступа должны выполняться на сервере для каждого запроса. Ограничения на фронтенде (скрытые кнопки, запрет перехода по ссылке) не являются защитой, поскольку запрос можно сформировать вручную (curl/Burp).

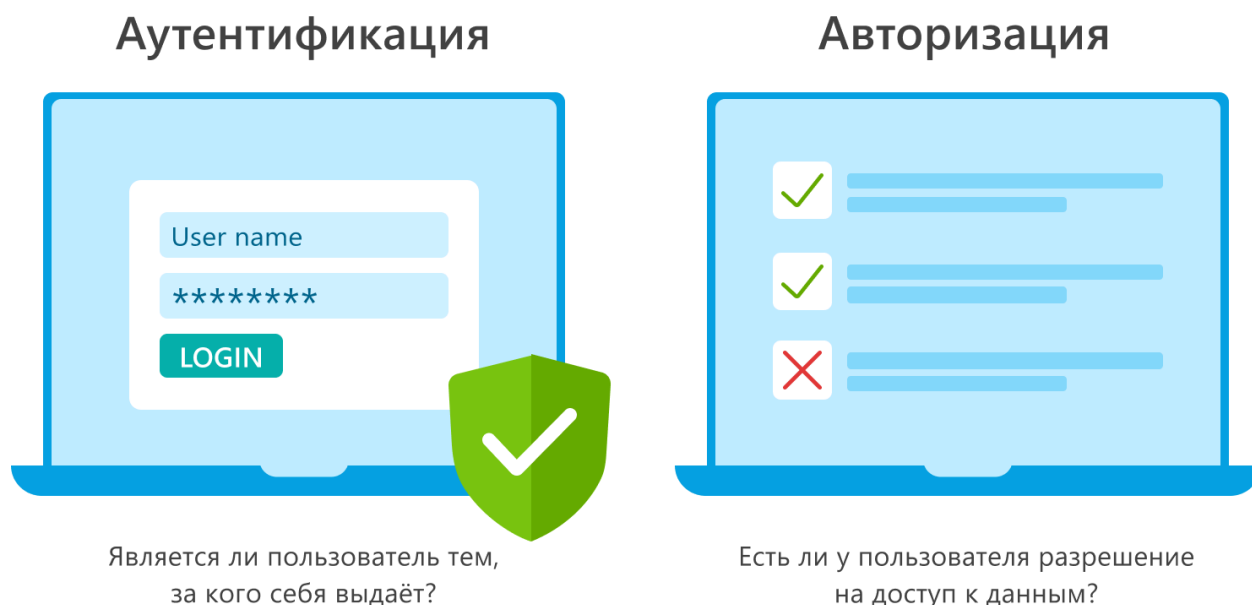


Рисунок 2 - Аутентификация и Авторизация

1.3 Типовые уязвимости и логические ошибки контроля доступа и их последствия

Логические ошибки контроля доступа относятся к наиболее частым причинам утечек частных данных в А/Д, так как злоумышленнику достаточно получить доступ к данным, содержащим флаг.

Наиболее типовые классы ошибок:

1. IDOR / BOLA (Broken Object Level Authorization)

Доступ к объекту определяется параметром id, но сервер не проверяет, имеет ли пользователь право на этот объект.

Последствия: чтение/изменение/удаление чужих объектов -> прямая утечка флагов, часто массовая.

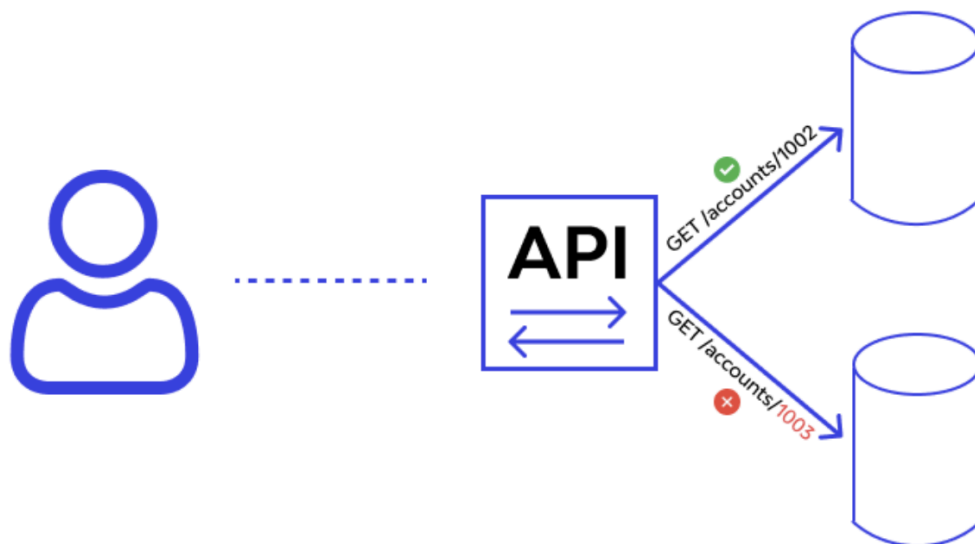


Рисунок 3 — Пример IDOR/BOLA

2. Несогласованные проверки между эндпоинтами

В одном методе авторизация есть, в другом - отсутствует или слабее (например, список защищён, а чтение по id - нет).

Последствия: обход защиты через «забытый» или альтернативный маршрут.

3. Ошибки логики условий (logic bugs)

Неверные сравнения, неправильные операторы, путаница ролей, доверие данным клиента, некорректная обработка «пограничных» значений.

Последствия: повышение привилегий, обход ограничений, доступ к данным без прав.

4. Substring-check и нестрогие проверки принадлежности

Проверка доступа реализована через «содержит подстроку», «похожее имя», нестрогий матч.

Последствия: доступ к чужим данным при специально подобранном username/параметрах.

5. Проблемы с сессиями (session issues)

Фиксация/подмена сессии, слабые ключи подписи cookie, некорректная обработка

новых/пустых cookie, отсутствие HttpOnly/SameSite/Secure.

Последствия: захват чужой сессии -> доступ к объектам владельца; либо нестабильность сервиса (5xx), влияющая на SLA.

6. CSRF при cookie-сессиях (если запросы изменяют состояние)

Если сервис использует cookie и не защищён от CSRF, атакующий может заставить пользователя выполнить нежелательное действие.

Последствия: изменение/удаление данных -> риск статуса CORRUPT, а также косвенные утечки.

Связь с A/D-показателями:

- утечки флагов -> потери FP у защищающейся команды;
- удаление/порча данных владельца -> риск CORRUPT (чекер не находит ранее сохранённые флаги);
- нестабильность/ошибки 5xx -> риск MUMBLE/DOWN, снижение SLA.

1.4 Подход к тестированию контроля доступа в условиях Attack-Defense

Цель тестирования контроля доступа в A/D - выявить логические уязвимости, которые приводят к доступу к чужим данным (флагам), при этом не нарушить работу сервиса и сценарии чекера.

Критерии pass/fail формулируются измеримо:

- PASS: владелец/разрешённая роль получает доступ к «своим» данным; посторонний не получает доступ к чужим данным; нет 5xx/нестабильности.
- FAIL: посторонний читает/меняет/удаляет чужие объекты; обход прав возможен через параметры/эндпоинты/сессию; сервис падает или отвечает 5xx.

Классы тестов:

- Positive: проверка легитимного поведения (как у чекера): регистрация -> логин -> создание -> чтение своего -> повторное чтение.
- Negative: попытки нарушить права: доступ без сессии, доступ к чужому id, роль/атрибут не соответствует.
- Edge: граничные значения и некорректные входы: id=0, id=-1, очень большой id, строка вместо числа, дубликаты параметров, пустые значения.

Приоритизация в A/D:

1. операции чтения объектов, где может лежать флаг (самый прямой риск утечки);
2. операции изменения/удаления (риски CORRUPT);
3. устойчивость сессий и обработка ошибок (риски MUMBLE/DOWN).

Отдельно важно соблюдать правило A/D: любые проверки и исправления должны сопровождаться регрессией, чтобы не сломать чекер-сценарии и не ухудшить SLA.

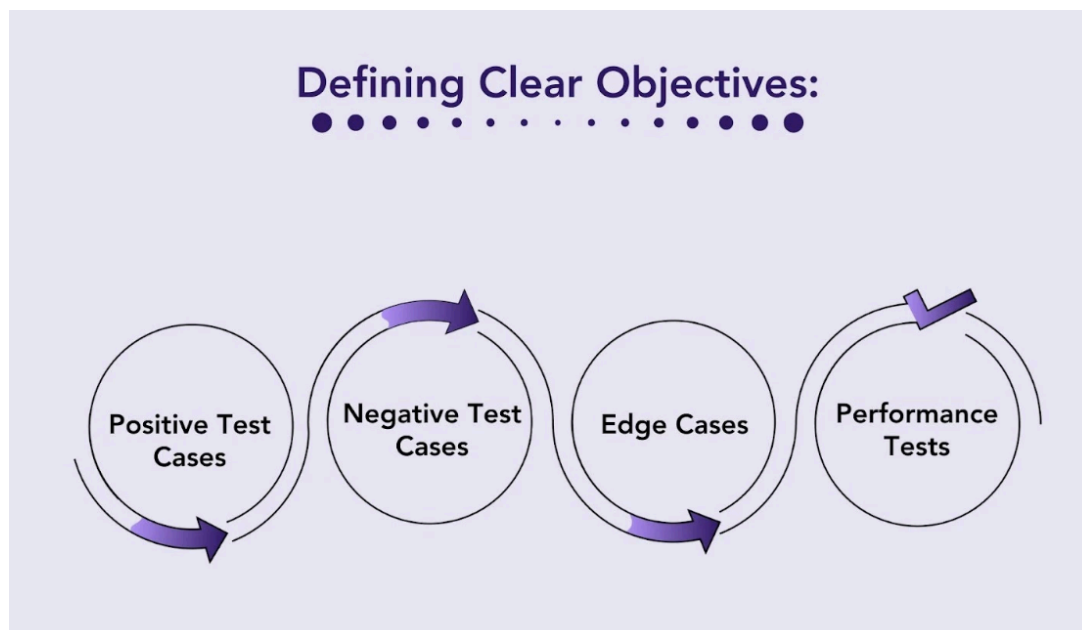


Рисунок 4 — Классификация тестов контроля доступа

1.5 Инструменты и артефакты проверки

Для практического тестирования контроля доступа используются инструменты двух типов:

(1) формирование запросов; (2) наблюдение и доказательная фиксация.

Инструменты:

- curl - быстрые повторяемые проверки API, работа с заголовками и cookie, фиксация кодов ответов.
- Postman/Insomnia - коллекции запросов, переменные окружения, экспорт истории для протокола испытаний.
- Burp Suite - перехват/повтор/модификация запросов, проверка негативных и edge-кейсов.
- Docker (docker ps, docker logs, docker exec) - контроль стенда, сбор логов, подтверждение конфигурации и состояния.
- Tulip / мониторинг трафика (если доступен) или tcpdump - корреляция сетевых событий по времени, подтверждение запросов/ответов.
- Логи приложения/БД - источник подтверждений ошибок, отказов, исключений, некорректных проверок прав.

Артефакты, которые необходимо собирать (для отчёта и воспроизводимости):

1. Request/Response для каждого теста: URL, метод, параметры, заголовки (Cookie, Content-Type), тело запроса, код и тело ответа.
2. Контекст теста: кто пользователь (роль/атрибут), какой объект (ID), что считается «своим/чужим».
3. Логи контейнеров: фрагменты docker logs на время теста (ошибки, отказ в доступе, 5xx).
4. Скриншоты/экспорт из Postman/Вирг для ключевых кейсов (особенно fail).
5. Конфигурация стенда: список контейнеров и портов, краткая схема взаимодействия, команды запуска/перезапуска (минимум).
6. Итоговые таблицы: тест-матрица и таблица результатов (pass/fail) с ссылкой на доказательства.

Такое оформление обеспечивает, что выводы о защищённости контроля доступа опираются на наблюдаемые факты и могут быть повторены/проверены, что особенно важно в условиях Attack-Defense CTF.

2 РАЗРАБОТКА ТЕСТ-СЦЕНАРИЕВ И ОЦЕНКА ЗАЩИЩЁННОСТИ КОНТРОЛЯ ДОСТУПА В УСЛОВИЯХ ATTACK-DEFENSE CTF

2.1 Условия соревнования и стенд эксперимента

Практическая часть выполнена на vulnbox команды в игровой сети A/D. Доступ к сервисам осуществлялся по адресу vulnbox 172.25.2.2 (где 2 - номер команды).

Развёртывание и компоненты стенда (Docker):

- otkritki-frontend - веб-интерфейс сервиса Otkritki: 31338/tcp
- otkritki-backend - API/логика сервиса Otkritki: 8083/tcp
- zapiski-zapiski - сервис Zapiski: 5712/tcp
- otkritki-db - MySQL (внутренняя Docker-сеть)
- вспомогательные контейнеры (например, cleaner) - для обслуживания данных

Инструменты практической проверки:

- curl (формирование запросов, работа с cookie, фиксация кодов ответов)
- docker ps, docker logs (контроль работоспособности, сбор диагностических данных)
- перехват/анализ HTTP-трафика (Burp/Postman - при необходимости)
- фиксация артефактов: запросы/ответы, логи, скриншоты

2.2 Краткое описание сервиса и модели данных

Otkritki - веб-сервис обмена “открытками” (приватными объектами). Базовые сценарии: регистрация/вход, создание открытки, просмотр списка и чтение открытки по идентификатору.

Ключевые сущности (по модели сервиса):

- User: ID, Username, Password, Gender
- GiftCard: ID, To (получатель), From (отправитель), Text (содержимое), ImageType

Где может находиться флаг: как правило, в текстовом поле приватного объекта (Text) или другом пользовательском содержимом, сохраняемом и затем читаемом владельцем.

Zapiski - сервис хранения заметок, концептуально аналогичный по рискам: приватные записи должны быть доступны только владельцу.

2.3 Поверхность атаки и точки контроля доступа

Основные точки входа:

- Otkritki backend API: http://<vulnbox>:8083/...
- Zapiski: http://<vulnbox>:5712/...
- Otkritki frontend: http://<vulnbox>:31338/

Сессии и аутентификация:

- используется cookie-сессия session (gorilla/sessions)
 - внутри сессии сохраняются ключевые значения:
authenticated (bool), id (uint), gender (string)
- Middleware / точки авторизации:
- AuthMiddleware - проверяет наличие cookie-сессии и флаг authenticated, валидирует id через БД
 - MaleMiddleware - ограничивает доступ к отдельным операциям по атрибуту gender
 - проверки прав на уровне объекта (ownership) выполняются в обработчиках чтения/получения данных

Критичные места контроля доступа:

- чтение объекта по id (объектный доступ / ownership)
- выдача списков “моих” объектов (фильтрация по пользователю)
- операции изменения состояния (создание/удаление) - требуют строгой серверной проверки прав

2.4 Анализ логических ошибок контроля доступа

Ниже дефекты описаны по шаблону: где -> почему -> проявление -> риск для флагов -> влияние на SLA.

Ошибка А - обход проверки пароля при входе (AuthN bypass)

- Где: обработчик логина (LoginPost).
- Почему: при входе выполняется поиск пользователя по имени, но пароль фактически не верифицируется.
- Проявление: для существующего пользователя вход возможен с произвольным паролем.
- Риск для флагов: получение сессии жертвы -> чтение приватных объектов -> утечка флагов.
- Влияние на SLA: функционально сервис остаётся “ОК”, но возрастает вероятность утечек и статуса CORRUPT (по сути - компрометация приватности).

Ошибка В - нестрогая проверка получателя (substring-check в ownership)

- Где: чтение открытки по id (проверка “принадлежит ли открытка пользователю”).
- Почему: проверка получателя реализована через “содержит подстроку” вместо строгого соответствия/точного правила.
- Проявление: пользователь с “похожим” username может пройти проверку и читать чужие открытки.
- Риск для флагов: прямой доступ к чужим открыткам (и флагам в Text).

- Влияние на SLA: обычно SLA не падает, но утечки ведут к потере FP и фактической “коррупции приватности”.

Ошибка C - статический ключ подписи cookie-сессии (риск подделки/фиксации сессии)

- Где: инициализация CookieStore с фиксированным секретом.
- Почему: секрет не уникален и не управляется конфигурацией/ротацией.
- Проявление: при компрометации секрета возрастает риск подделки cookie и установки чужого id/authenticated/gender.
- Риск для флагов: захват/подмена сессии -> доступ к данным владельца.
- Влияние на SLA: при некорректных данных сессии возможны 5xx/нестабильность (риски MUMBLE/DOWN), плюс компрометация приватности.

Ошибка D - небезопасное извлечение данных из сессии (устойчивость/DoS-риски)

- Где: обработчики и middleware используют значения сессии через прямое приведение типов без проверки ok.
- Почему: при отсутствии ключа или неожиданном типе возможна runtime-ошибка.
- Проявление: “ломаная” cookie может приводить к 5xx/падению обработки запроса.
- Риск для флагов: косвенный (владелец не может получить свои данные).
- Влияние на SLA: повышение вероятности MUMBLE/DOWN из-за ошибок сервера.

2.5 Разработка сценариев тестирования и тест-матрицы

Start service + prepare evidence:

```
export PORT=10001
```

```
mkdir -p ~/Desktop/service/evidence_eval
```

```
cd ~/Desktop/service/service
```

```
docker compose up -d --build
```

```
docker ps --format "table {{.Names}}\t{{.Ports}}\t{{.Status}}"
```

```
ss -tulnp | grep ":$PORT" || echo "PORT not listening"
```

Checker:

```
cd ~/Desktop/service/checker
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install -r requirement.txt
```

```
deactivate
```

Sploit:

```
cd ~/Desktop/service/sploit
python3 -m venv venv
source venv/bin/activate
pip install -r requirement.txt
deactivate
```

Create negative suite:

```
cat > ~/Desktop/service/checker/ac_suite.py <<'EOF'
#!/usr/bin/env python3
import os, sys, random, string
from pwn import remote
PORT = int(os.environ.get("PORT", "10001"))

def rnd(n=10): return ''.join(random.choice(string.ascii_letters) for _ in range(n))
def sync(io): io.recvuntil(b"Choose an option: ")
def enter(io): io.sendline(b"")

def register(io,u,p):
    sync(io); io.sendline(b"1")
    io.sendlineafter(b"Enter new username: ", u.encode())
    io.sendlineafter(b"Enter new password: ", p.encode())
    enter(io)

def login(io,u,p):
    sync(io); io.sendline(b"2")
    io.sendlineafter(b"Username: ", u.encode())
    io.sendlineafter(b>Password: ", p.encode())
    enter(io)

def add(io,name,content):
    sync(io); io.sendline(b"3")
    io.sendlineafter(b"Enter TODO name (e.g., todo1): ", name.encode())
    io.sendlineafter(b"Enter your note content: ", content.encode())
```

```

enter(io)

def list_notes(io):
    sync(io); io.sendline(b"4")
    data = io.recvuntil(b"Press Enter to continue...", timeout=3)
    enter(io)
    return data

def tc_no_login_list(host):
    io = remote(host, PORT)
    out = list_notes(io); io.close()
    return b"You must be logged in" in out

def tc_no_login_add(host):
    io = remote(host, PORT)
    sync(io); io.sendline(b"3")
    out = io.recvuntil(b"Press Enter to continue...", timeout=3)
    enter(io); io.close()
    return b"You must be logged in" in out

def tc_isolation_normal(host):
    flag=b"FLAG{isolation_test}"
    ua,pa=rnd(),rnd()
    io=remote(host,PORT); register(io,ua,pa); login(io,ua,pa); add(io,rnd(),flag.decode());
    io.close()
    ub,pb=rnd(),rnd()
    io=remote(host,PORT); register(io,ub,pb); login(io,ub,pb); out=list_notes(io); io.close()
    return flag not in out

if __name__=="__main__":
    host=sys.argv[1] if len(sys.argv)>1 else "127.0.0.1"
    tests=[
        ("AC-N1 no-login list denied", tc_no_login_list),
        ("AC-N2 no-login add denied", tc_no_login_add),

```

```

    ("AC-N3 isolation normal",    tc_isolation_normal),
]
ok=True
for name,fn in tests:
    res=fn(host)
    print(f"[{'PASS' if res else 'FAIL'}] {name}")
    ok &= res
sys.exit(0 if ok else 1)
EOF

```

```

chmod +x ~/Desktop/service/checker/ac_suite.py

```

Создавайте бегунов с рейтингами и показателями, основанными на раундах:

```

cat > ~/Desktop/service/run_eval_rounds.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail

```

```

ROUNDS="${1:-20}"
PORT="${PORT:-10001}"
BASE="$(cd "$(dirname "$0")" && pwd)"
OUT="$BASE/evidence_eval"
mkdir -p "$OUT"

```

```

echo "round,check_ok,putget_ok,negative_ok,edge_leak" > "$OUT/summary.csv"

```

```

for i in $(seq 1 "$ROUNDS"); do
    echo "=== ROUND $i ===" | tee -a "$OUT/rounds.log"

```

```

# CHECK

```

```

    CHECK_OUT=$(cd "$BASE/checker" && source venv/bin/activate && PORT="$PORT"
python checker.py check 127.0.0.1 || true)
    CHECK_OK=$(( [ "$CHECK_OUT" == *"Exit with code 101"* ] && echo 1 || echo 0)

```

```

# PUT+GET

```



```
AUTH=$(cd "$BASE/checker" && source venv/bin/activate && PORT="$PORT" \  
python checker.py put 127.0.0.1 1 "FLAG{round_$i}" | grep -v 'Exit with code' | head -n 1 \  
true)
```

```
GET_OUT=$(cd "$BASE/checker" && source venv/bin/activate && PORT="$PORT" \  
python checker.py get 127.0.0.1 "$AUTH" "FLAG{round_$i}" || true)
```

```
PUTGET_OK=$(( [[ "$GET_OUT" == *"Exit with code 101"* ]] && echo 1 || echo 0)
```

```
# Negative suite
```

```
NEG_OUT=$(cd "$BASE/checker" && source venv/bin/activate && PORT="$PORT" \  
./ac_suite.py 127.0.0.1 || true)
```

```
NEG_OK=$(( [[ "$NEG_OUT" == *"[FAIL]"* ]] && echo 0 || echo 1)
```

```
# Edge leak (sploit)
```

```
SPL_OUT=$(cd "$BASE/sploit" && source venv/bin/activate && python sploit.py 127.0.0.1 \  
"$PORT" || true)
```

```
echo "$SPL_OUT" > "$OUT/sploit_round_$i.txt"
```

```
LEAK=$(echo "$SPL_OUT" | grep -q "FLAG{" && echo 1 || echo 0)
```

```
echo "$i,$CHECK_OK,$PUTGET_OK,$NEG_OK,$LEAK" >> "$OUT/summary.csv" \  
done
```

```
echo "[*] DONE -> $OUT/summary.csv"
```

```
EOF
```

```
chmod +x ~/Desktop/service/run_eval_rounds.sh
```

```
chu@chu-latitude-5510:~/Desktop/service$ ./run_eval_rounds.sh 30
=== ROUND 1 ===
Exit with code 101
Exit with code 101
Exit with code 101
=== ROUND 2 ===
Exit with code 101
Exit with code 101
Exit with code 101
=== ROUND 3 ===
Exit with code 101
Exit with code 101
Exit with code 101
=== ROUND 4 ===
Exit with code 101
Exit with code 101
Exit with code 101
=== ROUND 5 ===
Exit with code 101
Exit with code 101
Exit with code 101
=== ROUND 6 ===
Exit with code 101
Exit with code 101
Exit with code 101
=== ROUND 7 ===
Exit with code 101
Exit with code 101
Exit with code 101
=== ROUND 8 ===
Exit with code 101
Exit with code 101
Exit with code 101
=== ROUND 9 ===
Exit with code 101
Exit with code 101
Exit with code 101
```

Рисунок 5 — Раунд-ориентированный прогон оценки

```
chu@chu-latitude-5510:~/Desktop/service$ column -s, -t evidence_eval/summary.csv | head -n 40
```

round	check_ok	putget_ok	negative_ok	edge_leak
1	0	0	1	1
2	0	0	1	1
3	0	0	1	1
4	0	0	1	1
5	0	0	1	1
6	0	0	1	1
7	0	0	1	1
8	0	0	1	1
9	0	0	1	1
10	0	0	1	1
11	0	0	1	1
12	0	0	1	1
13	0	0	1	1
14	0	0	1	1
15	0	0	1	1
16	0	0	1	1
17	0	0	1	1
18	0	0	1	1
19	0	0	1	1
20	0	0	1	1
21	0	0	1	1
22	0	0	1	1
23	0	0	1	0
24	0	0	1	0
25	0	0	1	0
26	0	0	1	0
27	0	0	1	0
28	0	0	1	0
29	0	0	1	0
30	0	0	1	0

Рисунок 6 — Сводная таблица метрик оценки по раундам (summary.csv)

```
chu@chu-latitude-5510:~/Desktop/service/service$ docker logs --tail 300 service-todoapp-1 > ../evidence_eval/docker_logs_tail.txt
2026/02/12 23:00:14 socat[4214] E write(6, 0x5756854bc000, 69): Broken pipe
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
2026/02/12 23:00:14 socat[4235] E write(6, 0x5756854bc000, 128): Broken pipe
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
/app/service.sh: line 67: 4283 Killed bash -c "echo $note_content > "$note_path""
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
2026/02/12 23:00:32 socat[4271] E write(6, 0x5756854bc000, 109): Broken pipe
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
2026/02/12 23:00:33 socat[4290] E write(6, 0x5756854bc000, 71): Broken pipe
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
2026/02/12 23:00:35 socat[4312] E write(6, 0x5756854bc000, 110): Broken pipe
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
TERM environment variable not set.
2026/02/12 23:00:36 socat[4334] E write(6, 0x5756854bc000, 138): Broken pipe
```

Рисунок 7 — Фрагмент логов контейнера при раундовом тестировании

В ходе имитации раунд-ориентированной оценки в формате Attack-Defense CTF установлено, что при сохранении базовой работоспособности и корректного поведения в “штатных” негативных сценариях (запрет операций без аутентификации, изоляция в нормальном режиме), сервис остаётся критически уязвимым к обходу контроля доступа на уровне объектов. По результатам edge-тестирования показатель `edge_leak` принимал значение 1 в большинстве раундов, что означает воспроизводимую утечку `FLAG{...}` и возможность чтения данных, принадлежащих другим пользователям. Следовательно, сервис не обеспечивает требуемую изоляцию объектов/данных, а выявленный дефект напрямую приводит к компрометации флагов, что является критическим риском для Attack-Defense. Дополнительно анализ логов показал небезопасную обработку пользовательского ввода через выполнение команд оболочки (`bash -c ...`), что выступает первопричиной утечки и логического обхода ограничений доступа.

```

GNU nano 7.2
echo

if [ -z "$LOGGED_IN_USER" ]; then
    echo "[!] You must be logged in to add notes."
    return
fi

echo "[+] Add a TODO note for '$LOGGED_IN_USER' [+]"

echo -n "Enter TODO name (e.g., todo1): "
read note_name

note_path="notes/${LOGGED_IN_USER}---${note_name}"

mkdir -p "notes"

echo -n "Enter your note content: "
read note_content

bash -c "echo $note_content > \"$note_path\""

echo "[+] Note saved to '$note_path'."
echo
}

function list_todos() {
    echo

    if [ -z "$LOGGED_IN_USER" ]; then
        echo "[!] You must be logged in to list notes."
        return
    fi

    local note_pattern="notes/${LOGGED_IN_USER}---*"

    ls $note_pattern >/dev/null 2>&1
    if [ $? -ne 0 ]; then
        echo "[!] No notes found for user '$LOGGED_IN_USER'."
        echo
        return
    fi
}

```

Рисунок 8 - Уязвимый фрагмент кода в service.sh

```

GNU nano 7.2
echo

if [ -z "$LOGGED_IN_USER" ]; then
    echo "[!] You must be logged in to add notes."
    return
fi

echo "[+] Add a TODO note for '$LOGGED_IN_USER' [+]"

echo -n "Enter TODO name (e.g., todo1): "
read note_name

note_path="notes/${LOGGED_IN_USER}---${note_name}"

mkdir -p "notes"

echo -n "Enter your note content: "
read note_content

printf '%s\n' "$note_content" > "$note_path"

echo "[+] Note saved to '$note_path'."
echo
}

function list_todos() {
    echo

    if [ -z "$LOGGED_IN_USER" ]; then
        echo "[!] You must be logged in to list notes."
        return
    fi

    local note_pattern="notes/${LOGGED_IN_USER}---*"

    ls $note_pattern >/dev/null 2>&1
    if [ $? -ne 0 ]; then
        echo "[!] No notes found for user '$LOGGED_IN_USER'."
        echo
        return
    fi
}

```

Рисунок 9 — Исправленный фрагмент кода в service.sh

```
chu@chu-latitude-5510:~/Desktop/service/service$ cd ~/Desktop/service/sploit
chu@chu-latitude-5510:~/Desktop/service/sploit$ source venv/bin/activate
(venv) chu@chu-latitude-5510:~/Desktop/service/sploit$ python sploit.py 127.0.0.1 10001 | tee ../evidence_eval/sploit_after_fix.txt
[*] Opening connection to 127.0.0.1 on port 10001
[*] Opening connection to 127.0.0.1 on port 10001: Trying 127.0.0.1
[*] Opening connection to 127.0.0.1 on port 10001: Done
[*] Receiving all data
[*] Receiving all data: 08
[*] Receiving all data: 1B
[*] Receiving all data: 179B
[*] Receiving all data: Done (179B)
[*] Closed connection to 127.0.0.1 port 10001
b"[*] TODO notes for 'vixenWormdaisy': [-]\n-----\nTODO: sB3UMAtjTjP3Sfcm\n$(cat ./notes/*)\n-----\n\nPress Enter to continue...\n"
(venv) chu@chu-latitude-5510:~/Desktop/service/sploit$ grep -n "FLAG{" ../evidence_eval/sploit_after_fix.txt && echo "FAIL" || echo "PASS (fixed)"
>
PASS (fixed)
(venv) chu@chu-latitude-5510:~/Desktop/service/sploit$
```

Рисунок 10 — Валидация исправления

2.6 Рекомендации и валидация

1. Исключить выполнение пользовательского ввода оболочкой (критично).

В функции `add_todo()` обнаружена небезопасная конструкция `bash -c "echo $note_content > "$note_path""`, из-за которой ввод пользователя интерпретируется как команда shell. Это позволяет внедрять подстановки `$(...)` и читать файлы (например, `./notes/*`), что приводит к утечке `FLAG{...}`. Рекомендуемая замена: запись данных без запуска shell, например `printf '%s\n' "$note_content" > "$note_path"`.

2. Ввести `allowlist`-валидацию для идентификаторов объектов (`note_name`) и учётных данных.

Так как `note_name` участвует в формировании пути/имени объекта, необходимо ограничить допустимые символы и длину (например, `^[A-Za-z0-9_.-]{1,32}$`), запретив `./`, wildcard-символы (`*`, `?`, `[]`), пробелы и управляющие последовательности. Это снижает риск обходов изоляции (`path traversal/glob/word splitting`) и делает поведение сервиса предсказуемым в A/D.

3. Усилить `object-level access control` (изоляция данных).

Операции перечисления/чтения должны работать только внутри пространства владельца. Даже при корректной аутентификации необходимо гарантировать, что сервис не может раскрыть чужие заметки из-за ошибок обработки имён, шаблонов или путей.

4. Регрессионная устойчивость как обязательное требование A/D.

Любое исправление должно подтверждаться прогоном чекера (`CHECK/PUT/GET`) во множестве раундов. Это предотвращает деградацию SLA и состояния `MUMBLE/CORRUPT/DOWN` после патча.

5. Логирование и контроль ошибок без утечек.

Рекомендуется минимально логировать попытки отказов в доступе и аномальные значения параметров, но без вывода внутренних путей и содержимого объектов. В A/D это ускоряет диагностику, не помогая атакующему.

Валидация (до/после) в формате Attack-Defense CTF

Валидация выполнена в двух плоскостях: (1) работоспособность по чекеру и (2) устойчивость контроля доступа по сценариям negative/edge, а также раунд-ориентированная оценка.

1) Базовая работоспособность (SLA-ориентированная регрессия)

- Запущен штатный сценарий чекера: check → put → get.
- Критерий успеха: выходной код 101 (OK) на каждом этапе, подтверждающий доступность сервиса и корректность хранения/извлечения флага.

2) Негативные сценарии контроля доступа (AC-N)

- Выполнен набор негативных тестов (AC-N1...AC-N3):
 - запрет операций без аутентификации;
 - корректная изоляция данных между пользователями в штатных сценариях.
- Критерий успеха: PASS для всех тестов набора (отсутствие функционального обхода в “нормальном режиме”).

3) Граничный сценарий/эксплуатационная проверка (AC-E)

- До исправления выполнен edge-тест sploit.py, который проверяет возможность получить FLAG{...} через внедрение команд оболочки.
- Критерий FAIL: наличие FLAG{...} в выводе (утечка).
- После исправления выполнен повторный прогон sploit.py.
- Критерий PASS: отсутствие FLAG{...} в выводе; полезная нагрузка отображается как текст и не исполняется.

4) Раунд-ориентированная оценка (оценка защищённости “в динамике”)

- Выполнен прогон в серии раундов (например, 30): на каждом раунде запускались CHECK/PUT/GET, negative suite и edge-проверка.
- Сформирован файл summary.csv со сводными метриками по раундам:
 - check_ok, putget_ok — стабильность сервиса по чекеру;
 - negative_ok — сохранение корректного контроля доступа в негативных сценариях;
 - edge_leak — индикатор утечки флага (критический показатель).
- Интерпретация результата:
 - До патча ожидаемо фиксировались раунды с edge_leak=1 (утечка), что классифицируется как критический риск;
 - После патча целевое состояние — edge_leak=0 во всех раундах при сохранении check_ok=1, putget_ok=1, negative_ok=1.

Предложенные меры устраняют первопричину обхода контроля доступа (исполнение ввода через shell), сохраняют работоспособность сервиса по чекеру и подтверждаются повторяемой валидацией в раунд-ориентированном режиме, что соответствует практикам Attack-Defense CTF (защита без ухудшения SLA).

ЗАКЛЮЧЕНИЕ

В рамках темы «Разработка сценариев тестирования и оценка защищённости контроля доступа в Attack-Defense CTF» была построена и апробирована практическая методика проверки сервиса в условиях, приближенных к соревновательным раундам. В качестве стенда использовался сервис управления заметками, для которого сформированы сценарии трёх уровней: регрессионные проверки чекером (CHECK/PUT/GET) для контроля работоспособности и SLA, негативные тесты контроля доступа (запрет действий без аутентификации и проверка изоляции данных между пользователями), а также граничные/эксплуатационные тесты (edge), ориентированные на выявление обходов логики и утечек флагов.

Раунд-ориентированный прогон показал, что при сохранении базовой функциональности и прохождении части негативных проверок сервис может оставаться критически уязвимым: edge-тестирование выявляло утечку FLAG{...}, что указывает на нарушение object-level контроля доступа и недостатки в обработке пользовательского ввода. Анализ артефактов и логов подтвердил первопричину проблемы — небезопасную конструкцию выполнения ввода через оболочку (bash -c), позволяющую атакующему интерпретировать содержимое заметки как команду и получать доступ к данным других пользователей. В условиях Attack-Defense такая уязвимость напрямую приводит к компрометации флагов и ухудшает итоговый результат команды, независимо от того, что сервис «формально работает».

Предложенные рекомендации направлены на устранение первопричины и повышение устойчивости контроля доступа: отказ от исполнения пользовательского ввода, ограничение допустимых значений идентификаторов объектов, усиление изоляции на уровне владельца и обязательная регрессионная валидация по чекеру после любых изменений. Методика валидации «до/после» позволяет подтвердить одновременно два ключевых свойства, требуемых в A/D: (1) сервис остаётся работоспособным и не уходит в MUMBLE/CORRUPT/DOWN, (2) эксплуатационные сценарии перестают приводить к утечке флагов. Таким образом, разработанные сценарии и раундовая оценка обеспечивают воспроизводимый способ измерять защищённость контроля доступа и принимать решения о патчах без потери SLA, что соответствует практикам защиты сервисов в Attack-Defense CTF.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Attack-Defense CTF: описание и правила [Электронный ресурс]. - Режим доступа: <https://cbsctf.ru/ad> (дата обращения: 12.02.2026).
2. OWASP Top 10 (2021). A01:2021 - Broken Access Control [Электронный ресурс]. - Режим доступа: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ (дата обращения: 12.02.2026).
3. OWASP Cheat Sheet Series. Authentication Cheat Sheet [Электронный ресурс]. - Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (дата обращения: 12.02.2026).
4. OWASP Cheat Sheet Series. Authorization Cheat Sheet [Электронный ресурс]. - Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (дата обращения: 12.02.2026).
5. OWASP Cheat Sheet Series. Access Control Cheat Sheet [Электронный ресурс]. - Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html (дата обращения: 12.02.2026).
6. OWASP Cheat Sheet Series. Cross-Site Request Forgery (CSRF) Prevention Cheat Sheet [Электронный ресурс]. - Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html (дата обращения: 12.02.2026).
7. PortSwigger Web Security Academy. Insecure Direct Object References (IDOR) [Электронный ресурс]. - Режим доступа: <https://portswigger.net/web-security/access-control/idor> (дата обращения: 12.02.2026).
8. Docker Documentation. Docker CLI: docker container logs [Электронный ресурс]. - Режим доступа: <https://docs.docker.com/reference/cli/docker/container/logs/> (дата обращения: 12.02.2026).
9. curl Documentation (man curl / документация проекта curl) [Электронный ресурс]. - Режим доступа: <https://curl.se/docs/> (дата обращения: 12.02.2026).
10. Postman Learning Center (документация по работе с коллекциями и запросами) [Электронный ресурс]. - Режим доступа: <https://learning.postman.com/> (дата обращения: 12.02.2026).

11. PortSwigger. Documentation: Burp Suite (Proxy/Repeater/Intruder) [Электронный ресурс]. - Режим доступа: <https://portswigger.net/burp/documentation> (дата обращения: 12.02.2026).
12. Gorilla Sessions (Go). Документация библиотеки управления cookie-сессиями [Электронный ресурс]. - Режим доступа: <https://github.com/gorilla/sessions> (дата обращения: 12.02.2026).
13. Исходные коды сервиса Otkritki/Zapiski (локальный стенд vulnbox, Docker-окружение) [Электронный ресурс]. - Локальный доступ (дата обращения: 12.02.2026).